

AI SMART MIRROR RETAIL SYSTEM

Complete Project Documentation - Current Development Stage

Project Owner: Aayush Khanal

Project Type: AI-powered retail catalogue, in-store navigation, and virtual try-on system

Current Stage: Catalogue interface completed; camera warning and try-on flow completed; live try-on engine prepared for interface integration

DOCUMENT OVERVIEW

- Project vision and current user experience
- Interface and folder architecture
- Catalogue and product data structure
- Map, privacy, and warning-screen rules
- Current live try-on engine and uploaded modules
- Performance, error handling, analytics, and future work

1. PROJECT OVERVIEW

The AI Smart Mirror Retail System is a privacy-first touchscreen retail platform designed for clothing stores.

The system starts as a large, responsive product catalogue rather than immediately opening the camera.

Customers can:

- Start from a welcome screen
- Browse departments such as Men, Women, Kids, Decors, Accessories, and Sale
- Open product categories such as Upper Fit, Lower Fit, Footwear, and Miscellaneous
- Browse products in a two-column vertically scrolling catalogue
- View product name, image, and price
- Open a product to view full details
- See sizes, availability, colour, discount status, and store location
- View “Complete Your Outfit” recommendations
- Open a store map and locate product sections
- Start virtual try-on only after accepting a camera usage warning
- Exit try-on and return safely to the product detail page

The camera, body tracking, clothing overlay, and AI processing remain inactive during normal catalogue browsing. This improves privacy, power usage, memory usage, and system responsiveness.

2. CURRENT PRODUCT VISION

The system is not only a virtual try-on mirror.

It is a retail kiosk and smart mirror platform that combines:

1. Product catalogue browsing
2. In-store navigation
3. Product and discount discovery
4. Outfit recommendations
5. Privacy-first virtual try-on
6. Future anonymous customer interaction analytics
7. Future sales prediction
8. Future outfit completion prediction

The commercial aim is to create a system suitable for retailers such as Primark, Sainsbury's clothing departments, independent clothing stores, and other high-footfall fashion retailers.

3. CURRENT USER EXPERIENCE FLOW

DISPLAY 1 — WELCOME SCREEN

Purpose:

- Introduce the system
- Keep the first interaction simple
- Avoid opening the camera

Main action:

- Start Shopping

Current privacy rule:

- The map button can be removed from the welcome screen
- No camera processing is active

DISPLAY 2 — DEPARTMENT SCREEN

Available options:

- Men
- Women
- Kids
- Decors
- Accessories
- Sale / Discounts

The user selects one department.

DISPLAY 3 — CATEGORY SCREEN

Example categories for Men:

- Upper Fit
- Lower Fit
- Footwear
- Accessories
- Miscellaneous
- Discounts

Upper Fit is used for shirts and tops.

Lower Fit is used for trousers, jeans, pants, and shorts.

DISPLAY 4 — PRODUCT CATALOGUE

Current catalogue behaviour:

- Vertical scrolling
- Two product columns
- Multiple rows
- Each product card displays only:
 - Product image
 - Product name
 - Product price

- Clicking or touching the full product card opens the product detail screen
- The entire card highlights as one object on hover
- Individual text or image areas do not highlight separately
- Discounted products display a simple DISCOUNT badge on the top-right of the image
- Product card dimensions remain fixed even when a discount badge is visible

Bottom-left:

- Map / Find in Store button

Bottom-right:

- Back button

DISPLAY 5 — PRODUCT DETAIL SCREEN

The product detail screen shows:

- Large product image
- Product name
- Price
- Colour
- Available sizes
- Availability status
- Product location
- Discount status
- Complete Your Outfit recommendation text

Current controls:

Top-right:

- Do Virtual Try On

Bottom-left:

- Map / Find in Store

Bottom-right:

- Back

The product description area uses clean text without visible text borders.

DISPLAY 6 — CAMERA WARNING SCREEN

The camera does not start immediately.

The warning screen explains:

- The camera is used only for live virtual try-on
- No photo or video is saved
- The camera turns off when try-on is exited
- The customer should stand fully visible before starting

Buttons:

- Cancel
- I Agree, Start Try-On

Privacy rule:

The Map button must not appear on:

- Camera warning screens
- Privacy screens
- Consent screens
- Payment screens
- Error screens
- Login/staff screens
- Other sensitive decision screens

DISPLAY 7 — TRY-ON SCREEN

Current interface flow:

- The selected product name is passed into the try-on screen

- The screen initially worked as a placeholder
- The next integration stage is to display the real live camera inside the PySide6 interface
- No map button is shown because the try-on screen is camera-sensitive
- Exit Try-On stops the camera and returns to the product detail screen

4. RECOMMENDED PROJECT FOLDER STRUCTURE

Interface/

```
|
|─ main.py
|─ product_catalog.py
|─ app_state.py
|
|─ data/
|   └─ catalogue.json
|
|─ img/
|   └─ shirts/
|      │ └─ shirt1.png
|      │ └─ shirt2.png
|      │ └─ shirt3.png
|      └─ shirt4.png
|   └─ pants/
|      │ └─ pant1.png
|      │ └─ pant2.png
|      │ └─ short1.png
|      └─ short2.png
|
|─ ui/
|   └─ __init__.py
|   └─ common_widgets.py
|   └─ welcome_screen.py
|   └─ department_screen.py
|   └─ category_screen.py
|   └─ catalogue_screen.py
|   └─ product_detail_screen.py
|   └─ camera_warning_screen.py
|   └─ map_screen.py
```



```
| └─ tryon_screen.py
|
| └─ tryon_engine/
|   ├── __init__.py
|   ├── tryon_engine.py
|   ├── main.py
|   ├── camera.py
|   ├── camera_thread.py
|   ├── bodytracking.py
|   ├── gesture_control.py
|   ├── clothing_overlay.py
|   ├── fit_engine.py
|   ├── segmentation.py
|   ├── overlay_ui.py
|   └─ config.json
```

Important:

- The interface root is the folder named Interface
- catalogue.json is stored in Interface/data/
- product images are stored in Interface/img/
- virtual try-on files are stored in Interface/tryon_engine/
- the standalone try-on main.py is kept only for debugging
- the PySide6 interface should use tryon_engine.py instead of opening the standalone OpenCV application

5. INTERFACE TECHNOLOGY

The catalogue interface uses:

- Python
- PySide6 / Qt for Python
- QMainWindow
- QStackedWidget
- QWidget
- QScrollArea
- QGridLayout
- QLabel
- QPushButton
- QPixmap

- QTimer for future live camera refresh

Why PySide6 is used:

- Better than OpenCV for touchscreen interfaces
- Supports smooth page navigation
- Supports scrolling
- Supports layouts
- Supports large retail-style buttons
- Supports image rendering
- Supports reusable components
- Integrates with existing Python virtual try-on code

OpenCV remains responsible for:

- Camera frames
- Image processing
- Clothing rendering
- Pose processing
- Alpha blending
- Virtual try-on rendering

6. APPLICATION NAVIGATION ARCHITECTURE

The main application uses a QStackedWidget.

Each display is a separate widget.

Example screen flow:

WELCOME

→ DEPARTMENT

→ CATEGORY

→ CATALOGUE

→ PRODUCT DETAIL

→ CAMERA WARNING

→ TRY-ON

Map can be opened from browsing and product screens.

The application stores:

- selected_department
- selected_category
- selected_product
- previous_screen_before_map

This allows the user to open the map and return to the correct previous screen.

7. COMMON MAP BUTTON LOGIC

The map button is created using:

ui/common_widgets.py

The shared function creates one consistently styled button:

- Map / Find in Store

This gives the same appearance on all permitted screens.

The map button should remain on:

- Department screen
- Category screen
- Catalogue screen
- Product detail screen
- Other browsing/navigation screens

The map button should not remain on:

- Camera warning
- Privacy warning
- Try-on camera mode
- Payment
- Error
- Consent
- Staff login
- Other sensitive screens

Changing the map button name in `common_widgets.py` updates it everywhere that uses `create_map_button()`.

8. PRODUCT CATALOGUE FORMAT

The current catalogue is stored in:

Interface/data/catalogue.json

Current product fields:

- id
- name
- department
- category
- price
- colour
- sizes
- available
- discount
- location
- image

- tryon_category
- fit

Example shirt product:

```
{
  "id": "shirt_001",
  "name": "Blue Casual T-Shirt",
  "department": "Men",
  "category": "Upper Fit",
  "price": "£11.99",
  "colour": "Blue",
  "sizes": ["S", "M", "L", "XL"],
  "available": true,
  "discount": false,
  "location": "Men Section / Upper Fit / Shirts Area",
  "image": "img/shirts/shirt1.png",
  "tryon_category": "Shirts",
  "fit": {
    "width_scale": 2.0,
    "height_scale": 1.2,
    "vertical_offset": 0.16,
    "horizontal_offset": 5
  }
}
```

Example pants product:

```
{
  "id": "pants_002",
  "name": "Blue Denim Jeans",
  "department": "Men",
  "category": "Lower Fit",
  "price": "£24.99",
  "colour": "Blue",
  "sizes": ["30", "32", "34", "36"],
  "available": true,
  "discount": false,
  "location": "Men Section / Lower Fit / Jeans Area",
}
```

```
"image": "img/pants/pant2.png",
"tryon_category": "Pants",
"fit": {
  "width_scale": 5.85,
  "height_scale": 1.45,
  "vertical_offset": 0.2,
  "horizontal_offset": 0
}
```

Important category logic:

Interface category:

- Upper Fit
- Lower Fit

Try-on category:

- Shirts
- Pants

This separation allows the retail interface and virtual try-on engine to use the same catalogue.

9. CURRENT PRODUCT INVENTORY

MEN — UPPER FIT

1. Blue Casual T-Shirt

- Price: £11.99
- Colour: Blue
- Sizes: S, M, L, XL
- Discount: No
- Image: img/shirts/shirt1.png

2. Red Summer Shirt

- Price: £14.99
- Colour: Red

- Sizes: S, M, L
- Discount: Yes
- Image: img/shirts/shirt2.png

3. White Slim Fit Top

- Price: £12.99
- Colour: White
- Sizes: S, M, L
- Discount: No
- Image: img/shirts/shirt3.png

4. Transparent T-Shirt

- Price: £9.99
- Colour: Transparent
- Sizes: S, M, L
- Discount: Yes
- Image: img/shirts/shirt4.png

MEN — LOWER FIT

1. Transparent Pants

- Price: £17.99
- Colour: Black
- Sizes: 30, 32, 34
- Discount: No
- Image: img/pants/pant1.png

2. Blue Denim Jeans

- Price: £24.99
- Colour: Blue
- Sizes: 30, 32, 34, 36
- Discount: No
- Image: img/pants/pant2.png

3. Black Summer Shorts

- Price: £12.99
- Colour: Black
- Sizes: S, M, L
- Discount: Yes
- Image: img/pants/short1.png

4. Miami easy Pink Shorts

- Price: £12.99
- Colour: easy Pink
- Sizes: S, M, L
- Discount: Yes
- Image: img/pants/short2.png

10. PRODUCT CATALOGUE DISPLAY LOGIC

The catalogue currently uses:

- Vertical scrolling
- Two columns
- Multiple rows
- Fixed card width and height
- Whole-card hover highlighting
- Product image
- Product name
- Product price

The catalogue does not show:

- Colour
- Sizes
- Availability
- Location
- Try-on button
- Product detail button

Full details appear only after touching or clicking the complete product card.

Discount badge logic:

- If discount is true, show DISCOUNT
- The badge is placed on the top-right of the image
- It is created as a child of the image label
- It does not change the product card size
- It does not reduce the image display area

11. PRODUCT DETAIL LOGIC

The product detail screen receives the selected product dictionary.

It displays:

- Product name
- Price
- Colour
- Sizes
- Availability
- Store location
- Discount status
- Product image
- Basic outfit completion suggestions

Example recommendation rules:

If selected product is a shirt:

- Recommend jeans
- Recommend shorts
- Recommend trainers
- Recommend lightweight jacket

If selected product is pants:

- Recommend shirts
- Recommend tops
- Recommend hoodie
- Recommend casual shoes

Current recommendations are rule-based text.

Future recommendation upgrade:

- Show real product cards
- Use product IDs
- Match colour
- Match category
- Match discount availability
- Learn from anonymous customer interactions

12. STORE MAP LOGIC

The map screen currently supports a simple section-based layout.

Example sections:

- You Are Here / Smart Mirror
- Men Upper Fit
- Men Lower Fit
- Discount Area
- Changing Room
- Checkout

When a product is selected, the map displays:

- Selected product name
- Product store location

Future map upgrades:

- Real store floor plan image
- Customer mirror position
- Product section highlight
- Walking route
- Multiple floors
- Accessibility route
- Live inventory location

- Discount section location
- Changing-room and checkout locations

13. CAMERA PRIVACY LOGIC

The system follows a privacy-first flow.

Camera remains off during:

- Welcome
- Department selection
- Category selection
- Catalogue browsing
- Product detail viewing
- Store map viewing

Camera starts only after:

1. User opens a product
2. User presses Do Virtual Try On
3. Camera warning appears
4. User presses I Agree, Start Try-On

Camera stops when:

- User exits try-on
- Try-on screen is closed
- Camera error occurs
- Application closes

No map button is shown on the camera warning or live try-on screen.

14. CURRENT LIVE TRY-ON ENGINE

The current live try-on engine contains:

- Threaded camera capture
- MediaPipe Pose
- MediaPipe Hands
- Body orientation detection
- Shirt overlay
- Pants overlay
- Shirt and pants together
- Product-specific fitting
- Fit smoothing
- Optional segmentation
- Gesture menu
- Pinch interaction
- Mouse interaction
- Freeze mode
- Debug mode
- Customer mode
- FPS display
- Live calibration
- Product-specific fit saving

The original standalone try-on application uses:

- `cv2.namedWindow()`
- `cv2.imshow()`
- `cv2.waitKey()`
- an infinite processing loop

This standalone file should remain available for debugging, but the interface version should not run it directly.

15. CAMERA THREAD

`camera_thread.py` uses:

- OpenCV VideoCapture

- Windows DirectShow backend
- Camera index 0
- Default resolution 960×540
- Target 30 FPS
- Background thread
- Thread-safe frame access
- Horizontal camera flip

The background thread continuously stores the latest frame.

Benefits:

- Better responsiveness
- Reduced camera blocking
- Faster frame retrieval
- Better separation between camera capture and UI rendering

16. CAMERA AND POSE PROCESSING

camera.py currently uses:

- MediaPipe Pose
- `model_complexity = 0`
- `smooth_landmarks = true`
- segmentation disabled in pose
- `detection confidence = 0.5`
- `tracking confidence = 0.5`
- pose processed every second frame
- previous landmarks reused between pose-processing frames

For interface integration, the camera must:

- Start only after consent
- Return frames instead of opening a window
- Release safely when exiting
- Support restart after returning to try-on

Recommended camera API:

- `start_camera()`
- `get_frame()`
- `release()`

17. BODY TRACKING

`bodytracking.py` currently provides:

- Landmark drawing
- Shoulder-centre calculation
- Body-centre drawing
- Front / side / back estimation
- Orientation stability filtering

Orientation detection uses:

- Shoulder distance
- Nose depth
- Stability across several frames

Current interface integration can initially force:

- FRONT mode

This is useful while improving front-facing try-on accuracy.

Later:

- Restore SIDE mode
- Restore BACK mode
- Use separate front/back product images
- Add side-view garment logic

18. FIT ENGINE

fit_engine.py calculates:

- Left shoulder
- Right shoulder
- Left hip
- Right hip
- Left knee
- Right knee
- Left ankle
- Right ankle
- Shoulder centre
- Hip centre
- Knee centre
- Ankle centre
- Shoulder width
- Hip width
- Torso height
- Leg height
- Waist position
- Crotch position

It also smooths body measurements to reduce clothing jitter.

Shirt fitting uses:

- Shoulder width
- Hip width
- Shoulder centre
- Torso height
- Width scale
- Height scale
- Vertical offset
- Horizontal offset

Pants fitting uses:

- Hip width
- Hip centre
- Leg height
- Width scale
- Height scale

- Vertical offset
- Horizontal offset

19. CLOTHING OVERLAY ENGINE

clothing_overlay.py currently supports:

- Catalogue loading
- Shirt product filtering
- Pants product filtering
- Product image loading
- Product-specific fit loading
- Product-specific fit saving
- Shirt selection
- Pants selection
- Shirt and pants together
- Remove shirt
- Remove pants
- Remove all
- Next product
- Previous product
- Shirt rendering
- Pants rendering
- Soft body shadow
- Alpha blending
- Pants inside-leg gap restoration
- Optional arm occlusion
- Optional leg occlusion
- Fit calibration controls

Important integration change:

The engine should use:

- tryon_category if available
- category as fallback

Example:

Upper Fit + Shirts

Lower Fit + Pants

The engine also needs a method such as:

`select_product_from_interface(selected_product)`

This method should:

- Match product ID, name, or image
- Load the correct product
- Detect whether it is a shirt or pants
- Load the product image
- Apply its fit values
- Reset smoothing
- Prepare it for rendering

20. CLOTHING RENDERING LOGIC

SHIRTS

The shirt renderer:

- Measures the body
- Calculates shoulder angle
- Clamps rotation
- Smooths angle, width, height, x, and y
- Resizes the shirt PNG
- Flips it for back mode
- Narrows it for side mode
- Rotates it
- Alpha blends it onto the frame
- Adds a soft body shadow

PANTS

The pants renderer:

- Measures hips and legs
- Calculates width and height
- Smooths width, height, x, and y
- Resizes the pants PNG
- Flips for back mode
- Narrows for side mode
- Alpha blends onto the frame
- Restores the inside-leg gap

21. SEGMENTATION

segmentation.py uses MediaPipe Selfie Segmentation.

Current settings:

- model_selection = 1
- threshold = 0.25
- segmentation processed every 3 frames
- previous mask reused for speed
- Gaussian blur applied to edges

The segmentation mask can keep clothing overlays inside the detected person area.

Current use:

- Optional
- Disabled by default for better FPS

Future use:

- Arm/garment occlusion
- Better edge control
- Person/background separation
- Improved realism
- Clothing behind hands and arms

22. GESTURE CONTROL

gesture_control.py currently supports:

- Maximum two hands
- Open-palm detection
- Pinch detection
- Hand processing every two frames
- Cached hand results
- Cached palm count
- Cached pinch position

Current gesture functions:

- Two palms toggle the menu
- Pinch selects menu options
- Pinch selects next-product arrow

Future interface decision:

The main catalogue uses touch.

Gesture control is more useful inside live try-on mode.

23. OVERLAY USER INTERFACE

overlay_ui.py is used inside the standalone OpenCV try-on engine.

It currently supports:

- Gesture menu
- Clothing groups
- Next-item arrow
- Current outfit information
- Developer fit information
- Customer mode
- Gesture feedback
- Fit editor

For the PySide6-integrated try-on version:

- Keep overlay_ui.py for debug mode
- Do not draw the entire retail catalogue through OpenCV
- Use PySide6 for navigation
- Use OpenCV overlays only inside the camera image where necessary

24. STANDALONE TRY-ON CONTROLS

Current keyboard controls in the standalone try-on debug application:

Q

- Quit

N

- Next product

P

- Previous product

R

- Remove all outfits

D

- Toggle debug landmarks

M

- Toggle gesture menu

G

- Toggle segmentation

Z

- Toggle freeze mode

T

- Save test frame

C

- Save global calibration

V

- Save current product fit into catalogue

U

- Toggle customer mode

W / S

- Increase / decrease width

L / J

- Increase / decrease height

I / K

- Move up / down

A / F

- Move left / right

25. CONFIGURATION FILE

config.json currently stores global fallback calibration.

Example fields:

- shirt_width_scale
- shirt_height_scale
- shirt_vertical_offset
- shirt_horizontal_offset
- pants_width_scale
- pants_height_scale
- pants_vertical_offset
- pants_horizontal_offset
- smoothing_factor
- arm_occlusion

Product-specific fit values in catalogue.json should take priority when a product is selected.

26. INTERFACE-TO-TRY-ON INTEGRATION

Recommended integration flow:

```
PySide6 product selected
→ selected product dictionary stored
→ Do Virtual Try On pressed
→ warning screen shown
→ user agrees
→ TryOnScreen.start_camera(product)
→ TryOnEngine.start(product)
→ camera starts
→ pose starts
→ clothing overlay loads selected product
→ processed frame returned
→ QTimer updates PySide6 image label
→ user exits
→ timer stops
→ engine stops
→ camera releases
→ return to product detail
```

The interface must not use:

- cv2.imshow()
- cv2.waitKey()

- the standalone infinite loop

Instead, the try-on engine should expose:

- start(selected_product)
- read_frame()
- stop()

27. CURRENTLY IMPLEMENTED VS NEXT INTEGRATION

IMPLEMENTED

- Welcome screen
- Department screen
- Category screen
- Product catalogue
- Vertical scrolling
- Two-column product grid
- Product cards with image, name, price
- Full-card touch/click behaviour
- Discount badge
- Product detail screen
- Basic recommendations
- Map screen
- Camera warning screen
- Try-on placeholder screen
- Privacy-sensitive screen rules
- Separate try-on engine folder
- Existing working standalone live try-on engine

NEXT INTEGRATION

- Add interface-compatible camera manager
- Add TryOnEngine wrapper
- Load selected product from interface
- Display real camera inside TryOnScreen
- Stop and restart camera safely
- Confirm shirt and pants loading through tryon_category
- Add user-facing camera error messages

28. ERROR HANDLING REQUIREMENTS

The commercial system should never crash directly in front of customers.

CATALOGUE ERROR

If catalogue.json is missing:

- Show “Catalogue temporarily unavailable”
- Log the exact file path
- Keep the application open

If JSON is invalid:

- Show a safe error page
- Do not display raw Python exceptions

IMAGE ERROR

If image is missing:

- Show No Image placeholder
- Keep the product card active if details still exist
- Disable virtual try-on if the try-on asset is unavailable

CAMERA ERROR

If camera fails:

- Show “Camera unavailable”
- Offer Retry
- Offer Back to Product
- Do not close the whole app

POSE ERROR

If body is not detected:

- Show “Please stand fully visible”
- Keep camera running

- Avoid crashing

TRY-ON PRODUCT ERROR

If product cannot be matched:

- Show “This item is temporarily unavailable for virtual try-on”
- Return safely to product details

MAP ERROR

If location is missing:

- Show “Location unavailable”
- Keep general map available

29. PERFORMANCE PRINCIPLES

To keep the system smooth:

- Camera remains off during catalogue browsing
- MediaPipe remains unloaded until try-on starts
- Product thumbnails are used in catalogue cards
- Full-size images are loaded only on product detail
- Pose runs every two frames
- Gesture detection runs every two frames
- Segmentation runs every three frames
- Previous results are reused
- Camera capture runs in a separate thread
- PySide6 updates camera using QTimer
- Avoid blocking loops in the UI thread
- Release all camera and model resources on exit

30. FUTURE ANALYTICS DESIGN

Future analytics should collect only privacy-conscious interaction data.

Suitable anonymous events:

- Anonymous session ID
- Department opened
- Category opened
- Product viewed
- Product tried on
- Map opened
- Discount viewed
- Recommendation selected
- Time spent on screen
- Product added or saved
- Try-on completed
- Try-on exited

Avoid collecting:

- Name
- Face image
- Stored video
- Exact identity
- Skin colour
- Ethnicity
- Religion
- Health data
- Private body images

Possible future use:

- Predict product demand
- Predict future sales
- Find popular categories
- Find products frequently viewed together
- Recommend likely outfit completions
- Identify products customers view but do not buy
- Measure discount visibility
- Improve store section placement

31. FUTURE REALISM UPGRADES

Realistic colour:

- Controlled product photography
- Plain neutral background
- Fixed lighting
- Colour calibration card
- White balance correction
- Product-specific colour metadata
- Display colour calibration

Realistic fitting:

- Better garment masks
- Better body segmentation
- Mesh warping
- Depth estimation
- Front/back/side garment assets
- Occlusion around arms and hands
- Better shoulder and hip fitting
- Product-specific anchor points
- 3D garment simulation later

32. HARDWARE DIRECTION

Prototype:

- Windows PC or laptop
- RTX 3060
- Webcam
- Standard monitor or touchscreen

Commercial direction:

- Large high-brightness display
- Optional two-way mirror glass
- IR touch frame for touch through glass
- Camera hidden in top frame
- Mini PC or GPU-enabled computer
- Kiosk enclosure
- Cooling and cable management
- Automatic startup

- Watchdog recovery
- Remote product updates

33. CURRENT DEVELOPMENT PRIORITY

Priority 1:

- Make live camera appear inside PySide6 TryOnScreen

Priority 2:

- Pass selected product from catalogue to try-on engine

Priority 3:

- Load correct shirt or pants automatically

Priority 4:

- Stop camera safely on exit

Priority 5:

- Add friendly camera and product error handling

Priority 6:

- Improve fitting and realism after integration is stable

34. IMMEDIATE NEXT STEPS

1. Create Interface/tryon_engine/__init__.py

2. Keep uploaded live try-on files inside Interface/tryon_engine/

3. Modify camera.py so camera starts only through start_camera()

4. Modify clothing_overlay.py so:

- catalogue loads from Interface/data/catalogue.json
- images load from Interface/img/
- config loads from Interface/tryon_engine/config.json
- tryon_category is used
- select_product_from_interface() is added

5. Create tryon_engine/tryon_engine.py

6. Replace placeholder ui/tryon_screen.py with live-frame version

7. Update interface/main.py:

- start_virtual_try_on()
- exit_virtual_try_on()

8. Test one shirt first

9. Test one pair of pants

10. Test exit and restart camera

11. Add user-facing errors

12. Then improve realism

35. FINAL CURRENT PROJECT DESCRIPTION

The AI Smart Mirror Retail System is a privacy-first touchscreen retail platform that combines a department-based product catalogue, vertically scrolling product discovery, store-map navigation, discount visibility, product details, outfit recommendations, and an optional virtual try-on experience.

The system keeps the camera and AI models inactive until the customer explicitly selects a product, chooses virtual try-on, and accepts the camera notice. The selected product is then passed into a separate OpenCV and MediaPipe try-on engine, allowing the retail interface and AI rendering engine to remain modular, easier to debug, and more suitable for commercial development.